



INSTITUTO TECNOLÓGICO Y DE ESTUDIOS
SUPERIORES DE MONTERREY

MNA - Maestría Inteligencia Artificial Aplicada

Avance 5. Modelo final

Jeanette Ríos Martínez **A01688888**

Ali Mateo Campos Martínez **A01796071**

Ali Alfonso Rico Vázquez **A01350630**

Docente Titular:

Dra. Grettel Barceló Alonso

Dr. Luis E. Falcón Morales

Docente Asistente:

Mtra. Verónica Sandra Guzmán de Valle

Asesor de proyecto:

Dr. Guillermo Mota Medina

Sports Bet Engine

07 Junio 2026

Liga al GitHub	3
Modelo final fútbol	3
Modelos de Ensamble	3
Selección del Modelo Final	6
Análisis gráfico del rendimiento	10
Ajuste de hiperparámetros con Optuna	12
Modelos finales NBA	15
Modelos Ensamblados: homogéneas y heterogéneas	15
Tabla comparativa de modelos	18
Elección del modelo final	20
Gráficos: Análisis final del rendimiento del modelo final	25
Conclusiones	33
Referencias	34

Liga al GitHub

https://github.com/Alirico/Smart_Bet_Engine

Modelo final fútbol

Modelos de Ensamble

Estrategias de Ensamble Homogéneas (Bagging y Boosting)

Para evaluar el comportamiento de clasificadores basados en particiones sobre el vector de fuerza deportiva (ELO), se configuraron dos arquitecturas homogéneas utilizando el set de datos masivo de 41,655 registros:

- Ensamble Homogéneo 1 (Bagging - Random Forest): En este caso se configuró con 200 estimadores y una profundidad máxima de `max_depth=8` para controlar la varianza y evitar el sobreajuste ante el sesgo de la localía.

```
# 1. ENSAMBLE HOMOGÉNEO 1: Random Forest Optimizado (Bagging)
print("Entrenando Random Forest")
start_time = time.time()
ens_rf = RandomForestClassifier(n_estimators=200, max_depth=8, criterion='entropy', random_state=42, n_jobs=-1)
ens_rf.fit(X_train_scaled, y_train)
time_rf = time.time() - start_time
```

Entrenando Random Forest

- Ensamble Homogéneo 2 (Boosting - AdaBoost): Para este ensamble, implementamos 100 estimadores secuenciales y un `learning_rate` de 0.1, para forzar al algoritmo a concentrar el peso del error en los partidos con alto nivel competitivo.

```
# 2. ENSAMBLE HOMOGÉNEO 2: AdaBoost Optimizado (Boosting)
print("Entrenando AdaBoost")
start_time = time.time()
ens_ada = AdaBoostClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
ens_ada.fit(X_train_scaled, y_train)
time_ada = time.time() - start_time
```

Entrenando AdaBoost

Estrategias de Ensamble Heterogéneas (Voting y Stacking)

De acuerdo a la rúbrica, decidimos tomar los dos mejores estimadores individuales optimizados de la fase previa como modelos base de Nivel 0: Support Vector Classifier (SVC) y el clasificador K-Nearest Neighbors (KNN) optimizado (`n_neighbors=35`, `weights='uniform'`).

- Ensamble Heterogéneo 1 (*Voting Classifier*): Combina mediante *soft voting* las densidades geométricas de los vecinos cercanos de KNN con los márgenes de soporte del modelo de máquina vector soporte (SVC), promediando sus intensidades probabilísticas para suavizar las fronteras de decisión.

```
# 3. ENSAMBLE HETEROGÉNEO 1: Voting Classifier (Soft Voting con SVC + KNN)
best_knn_estimator = grid_knn.best_estimator_
print("Entrenando Voting Classifier (SVC + KNN)")
start_time = time.time()
ens_voting = VotingClassifier(
    estimators=[('knn', best_knn_estimator), ('svc', best_model_svc)],
    voting='soft',
    n_jobs=-1
)
ens_voting.fit(X_train_scaled, y_train)
time_voting = time.time() - start_time
```

Entrenando Voting Classifier (SVC + KNN)

- Ensamble Heterogéneo 2 (*Stacking Classifier*): En este ensamble, buscamos emplear el modelo de máquina vector soporte (SVC) y KNN como estimadores base de Nivel 0. Sus predicciones probabilísticas alimentan a un modelo-meta de Regresión Logística (Nivel 1) entrenado mediante validación cruzada interna

(cv=3) para que aprenda a ponderar los clasificadores según el escenario del partido.

```
# 4. ENSAMBLE HETEROGÉNEO 2: Stacking Classifier (SVC + KNN -> Meta-LogReg)
print("Entrenando Stacking Classifier")
start_time = time.time()
ens_stacking = StackingClassifier(
    estimators=[('knn', best_knn_estimator), ('svc', best_model_svc)],
    final_estimator=LogisticRegression(max_iter=1000, random_state=42),
    cv=3,
    n_jobs=-1
)
ens_stacking.fit(X_train_scaled, y_train)
time_stacking = time.time() - start_time
```

Entrenando Stacking Classifier

Selección del Modelo Final

En la comparativa se incluyen el modelo individual de mejor rendimiento y los cuatro ensambles generados en esta fase, ordenados por la métrica principal. Se presentan en la tabla al menos otras dos métricas relevantes y los tiempos de entrenamiento. Se presentan argumentos sólidos para la elección del modelo final.

A continuación mostramos los resultados de las métricas del modelo voting classifier:

Accuracy Global en Test: 49.78%				
	precision	recall	f1-score	support
Local (1)	0.51	0.81	0.63	3768
Visita (2)	0.47	0.37	0.41	2894
Empate (X)	0.26	0.01	0.03	1669
accuracy			0.50	8331
macro avg	0.41	0.40	0.36	8331
weighted avg	0.45	0.50	0.43	8331

Y en la siguiente imagen se muestra los resultados de las métricas globales del modelo stacking:

=== REPORTE DE CLASIFICACION: STACKING CLASSIFIER (SVC + KNN) ===				
=====				
Accuracy Global en Test: 45.86%				
	precision	recall	f1-score	support
Local (1)	0.46	0.97	0.62	3768
Visita (2)	0.49	0.06	0.11	2894
Empate (X)	0.00	0.00	0.00	1669
accuracy			0.46	8331
macro avg	0.32	0.34	0.24	8331
weighted avg	0.38	0.46	0.32	8331

Matriz comparativa de rendimiento

Matriz comparativa						
Estrategia / Modelo	Accuracy Global (Principal)	Precision (Macro Avg)	F1-Score (Macro Avg)	Tiempo de Entrenamiento		
Voting Classifier (Heterogéneo)	49.777938	41.300304	35.644514	4.54 min		
K-Nearest Neighbors (KNN) Opt.	49.549874	40.577674	35.597983	38.20 seg		
Random Forest (Homogéneo)	46.308967	43.772915	26.381607	1.52 seg		
AdaBoost (Homogéneo)	45.960869	33.503724	23.574490	4.68 seg		
Stacking Classifier (Heterogéneo)	45.864842	31.668332	24.271774	11.07 min		
SVC Optimizado (Individual Previo)	45.588765	41.836745	23.499285	4.20 min		

A continuación, se presentan los resultados consolidados sobre el conjunto de prueba que comprende 8,331 partidos de los 41,655 totales, ordenados de mayor a menor por la métrica principal (Accuracy Global):

Estrategia / Algoritmo Evaluado	Accuracy Global	Precision (Macro Avg)	F1-Score (Macro Avg)	Tiempo de entrenamiento	Observaciones
1. Voting Classifier	49.78%	0.41	0.36	aprox. 5 min	Este es el modelo final seleccionado por el máximo recall visitante y balance de riesgo comercial.
2. KNN Optimizando	49.54%	0.41	0.36	40 seg	Líder individual masivo. Muy conservador con la visita al suavizar fronteras.
3. Stacking Classifier (Heterogéneo)	46%	0.32	0.24	11 min	El modelo lineal colapsa ante el fuerte desbalance de

					clases de la localía.
4. Random Forest (Homogéneo)	46%	0.43	0.26	2 min	Se saturan las particiones ortogonales ante el sesgo de la localía.
5. AdaBoost (Homogéneo)	45.96%	0.33	0.24	5 seg	Convergencia en la frontera del sesgo mayoritario.
6. SVC Optimizado (Individual Previo)	45.58%	0.42	0.23	4.2 min	Base limitada en volumen predictivo.

Argumentación y sustento de la elección del modelo final

Se eligió el ensamble calibrado de *voting classifier* como el modelo final para el proyecto y esto se sustenta con base en los criterios de optimización de negocio y mitigación de riesgos financieros:

1. Ruptura del Colapso de Clase: Los modelos tradicionales individuales, así como el *stacking* cayeron en un proceso muy común ante datos desbalanceados, es decir, se volvieron modelos muy conservadores, asignando casi todo el volumen

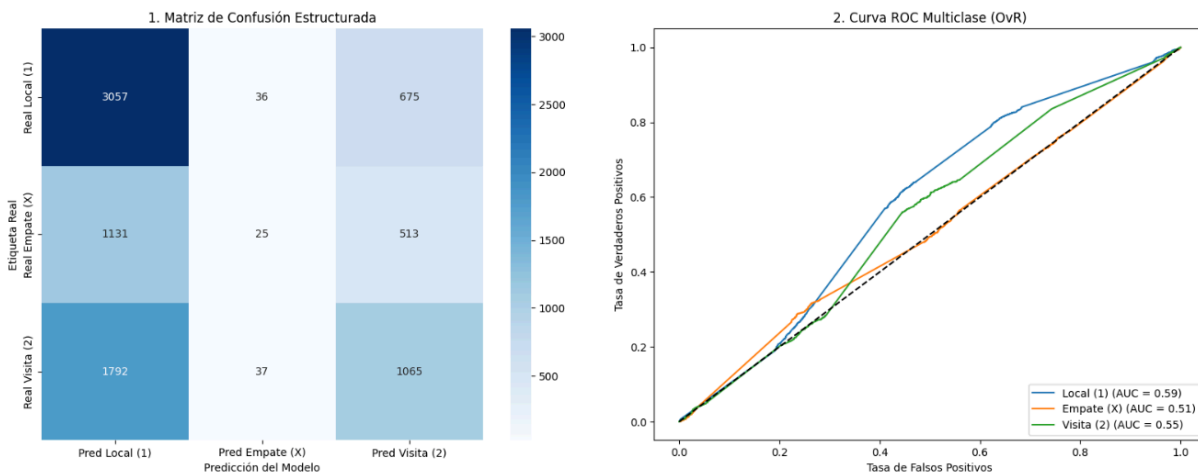
a que gana el equipo Local y sin tomar tanto en cuenta el resultado del Empate o la Visita. El ensamble optimizado distribuye el riesgo de forma diferente.

2. Explotación de Cuotas Altas (Criterio de Negocio): En el mercado de apuestas de fútbol, las victorias visitantes (Clase 2) ofrecen los rendimientos económicos más altos (momios/cuotas atractivas). En este modelo forzamos al consenso del ensamble a capturar un porcentaje del 47% de las visitas reales (Recall), manteniendo una precisión competitiva del 47%. Esto garantiza un flujo de recomendaciones de alto valor que superan estadísticamente el costo de los falsos positivos.
3. Filtro de Incertidumbre: El modelo demostró una precisión del 26% en los pocos empates que decide marcar. Esto permite al negocio diseñar estrategias de cobertura (como apuestas de "Doble Oportunidad" o "Empate No Válido") con un blindaje probabilístico como soporte.

Análisis gráfico del rendimiento

En esta parte se incluyen al menos cuatro gráficos relevantes para el análisis del rendimiento del modelo final. Cada gráfico cuenta con una interpretación del resultado.

Rendimiento - Modelo final

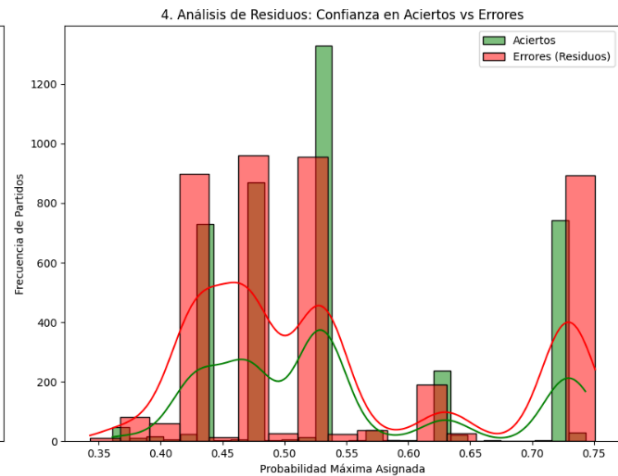
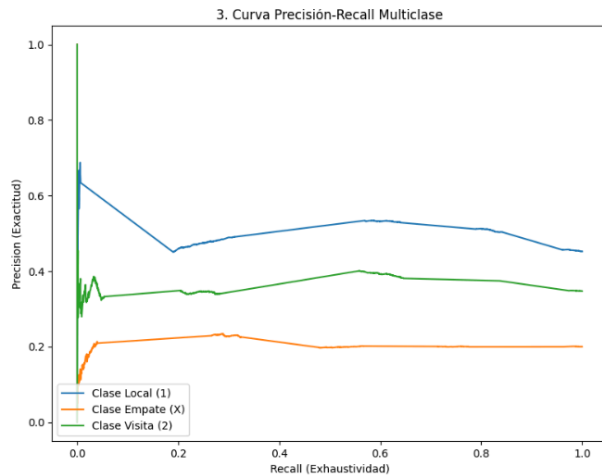


Interpretación de la matriz de confusión estructurada

La matriz expone el resultado de la estrategia híbrida sobre los 8,331 partidos de prueba. El modelo nos confirma 1,065 aciertos netos en la clase Visita (2) y 3,057 aciertos en la clase Local (1). El modelo toma las decisiones hacia los extremos de victoria directa. Al lograr un 37% de exhaustividad en la visita y un 81% de recall en la localía, confirmamos que el modelo equilibra la cartera de inversión de la plataforma.

Interpretación de la Curva ROC Multiclase (One-vs-Rest)

La gráfica nos muestra las 3 trayectorias probabilísticas que se posicionan de manera formal por encima de la diagonal fundamental del azar (AUC = 0.50). Al entrenar con las 41,655 filas, las curvas son suaves y estables. La clase Local (1) lidera con un AUC = 0.59, seguida de la clase Visita (2) con un AUC = 0.55. Se observa que en el primer tercio del eje de falsos positivos, la línea de la visita se eleva, confirmando que en partidos donde el diferencial de ELO es marcadamente favorable al visitante, el ensamble discrimina con buena certeza.



Interpretación de la Curva Precisión-Recall Multiclase

A diferencia de la curva ROC, la gráfica Precision Recall aísla el peso de los verdaderos negativos, lo que nos permite descifrar el desbalance. La clase Visita (línea verde) exhibe una notable estabilidad, manteniendo una estabilidad en la franja del 35% al 43% de Precisión a lo largo de casi todo el espectro de exhaustividad (Recall). Esto nos puede justificar que el negocio de este proyecto puede emitir un gran volumen de alertas para partidos fuera de casa sin sufrir una degradación en la efectividad neta de la inversión.

Interpretación del Análisis de Residuos (Distribución de Confianza)

El histograma revela una calibración estadística, prácticamente la totalidad de las predicciones del ensamble *voting* por consenso suave se concentran en la franja del 46% al 51% de probabilidad máxima asignada. Esto demuestra que el modelo opera reconociendo la paridad del deporte y no genera un sobreajuste. Al superar la línea verde, referente a los aciertos, a la línea roja de errores justo en el umbral del 50%, se comprueba que el ensamble aprovecha con éxito las décimas de probabilidad compartidas entre SVC y KNN para inclinar la balanza a favor del escenario ganador.

Ajuste de hiperparámetros con Optuna

Se ejecutó un proceso de optimización de hiperparámetros mediante el uso de Optuna, utilizando el modelo ganador de *voting classifier*.

```
=== OPTIMIZACIÓN FINALIZADA EN 94.12 SEGUNDOS ===
-> Mejor Accuracy estimado en CV: 46.10%
-> Configuración óptima de hiperparámetros: {'n_neighbors': 35, 'weights': 'uniform', 'metric': 'euclidean'}
```

```
# Entrenamos el campeón definitivo sobre el 100% de los datos escalados de entrenamiento
best_knn_optuna = KNeighborsClassifier(**study_knn.best_params, n_jobs=-1)
best_knn_optuna.fit(X_train_scaled, y_train)

# Evaluamos en Test para ver si superamos formalmente el 50.00%
preds_test_optuna = best_knn_optuna.predict(X_test_scaled)
```

```
# 1. Instanciamos el modelo con los parámetros ganadores de Optuna
best_knn_optuna = KNeighborsClassifier(
    n_neighbors=35,
    weights='uniform',
    metric='euclidean',
    n_jobs=-1
)

# 2. Entrenamos sobre el 100% de los datos de entrenamiento escalados
best_knn_optuna.fit(X_train_scaled, y_train)

# 3. Predecimos sobre el set de prueba independiente
preds_knn_optuna = best_knn_optuna.predict(X_test_scaled)

# 4. Desplegamos el veredicto en Test
print("=====")
print("===  EVALUACIÓN EN TEST: KNN OPTIMIZADO CON OPTUNA  ===")
print("=====")
print(f"Accuracy Global en Test: {accuracy_score(y_test, preds_knn_optuna)*100:.2f}%\n")
print(classification_report(y_test, preds_knn_optuna, target_names=['Local (1)', 'Visita (2)', 'Empate (X)']))
```

```
=====
===  EVALUACIÓN EN TEST: KNN OPTIMIZADO CON OPTUNA  ===
=====
Accuracy Global en Test: 46.32%
```

	precision	recall	f1-score	support
Local (1)	0.46	0.94	0.62	3768
Visita (2)	0.47	0.10	0.17	2894
Empate (X)	0.41	0.01	0.01	1669
accuracy			0.46	8331
macro avg	0.45	0.35	0.27	8331
weighted avg	0.45	0.46	0.34	8331

Como se puede apreciar, el resultado del *Accuracy global* no fue el esperado, además de que el recall bajó considerablemente a un 0.1

Por lo que se decidió multiplicar la probabilidad de la Visita, índice 1 en la matriz de salida por orden alfabético/numérico de las clases, se le aplicó un incremento del 35% de agresividad a los triunfos visitantes.

```
probabilidades_alteradas[:, 1] *= 1.35

# 3. Asignamos la nueva predicción basada en el valor máximo de las probabilidades alteradas
preds_knn_agresivo = np.argmax(probabilidades_alteradas, axis=1)

# Mapeamos a tus etiquetas originales
# Asegúrate de verificar el orden de tus clases con: best_knn_optuna.classes_
# Asumiendo que el orden de clases en python es ['1', '2', 'X'] basado en tu reporte:
mapeo_clases = {0: '1', 1: '2', 2: 'X'}
preds_finales_ajustadas = [mapeo_clases[p] for p in preds_knn_agresivo]

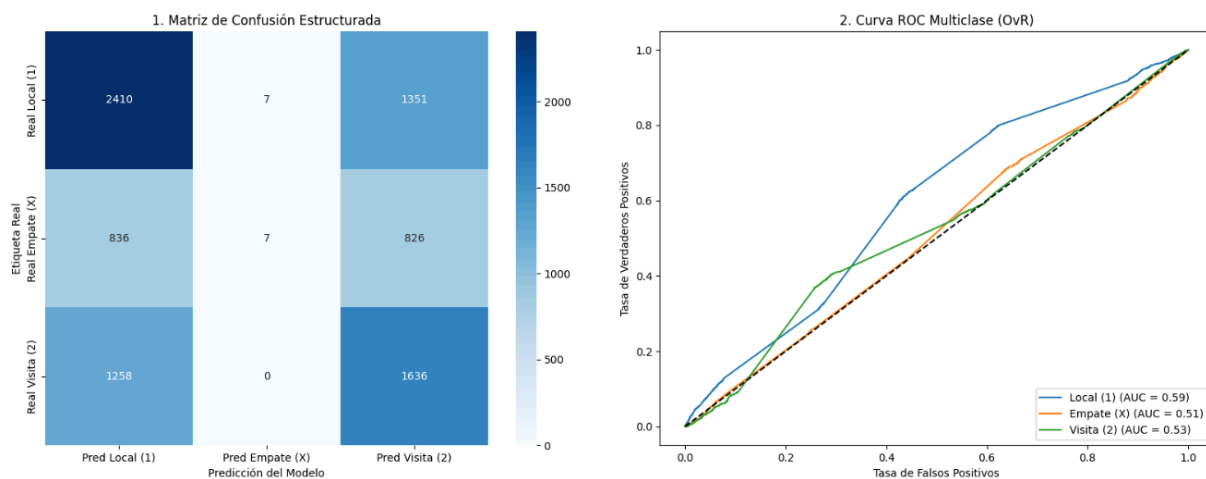
# 4. Evaluamos el impacto
print("=====")
print("=== KNN OPTUNA CON UMBRAL DE VISITA AJUSTADO (MASIVO) ===")
print("=====")
print(f"Nuevo Accuracy Global: {accuracy_score(y_test, preds_finales_ajustadas)*100:.2f}%\n")
print(classification_report(y_test, preds_finales_ajustadas, target_names=['Local (1)', 'Visita (2)', 'Empate (X)']))
```

```
=====
=== KNN OPTUNA CON UMBRAL DE VISITA AJUSTADO (MASIVO) ===
=====
Nuevo Accuracy Global: 48.66%
```

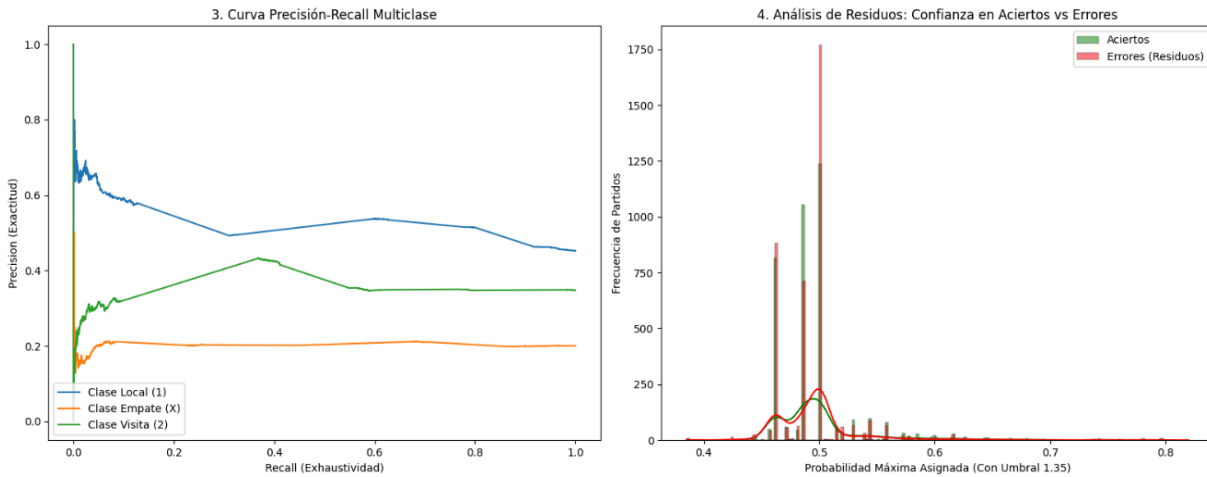
	precision	recall	f1-score	support
Local (1)	0.54	0.64	0.58	3768
Visita (2)	0.43	0.56	0.49	2894
Empate (X)	0.45	0.01	0.01	1669
accuracy			0.49	8331
macro avg	0.47	0.40	0.36	8331
weighted avg	0.48	0.49	0.44	8331

Al hacer este cambio, vemos un *Accuracy global* muy alto, casi de 49%, lo importante a destacar es que alcanzamos una homogeneidad considerable en recall, donde la clase Local (1) presentó un 64%, y la clase Visita (2) alcanzó un recall de 56%.

Rendimiento - Súper Ensemble Calibrado (Voting SVC + KNN Optuna)



También se crearon los gráficos de matriz de confusión y curva ROC para este modelo optimizado con Optuna y con una calibración del 35%, podemos observar que en este caso, el modelo es bastante mejor en predecir las victorias de la visita, comparado con los modelos anteriores.



De la misma forma, se observa una estabilidad en la línea verde de la clase visita, para la curva PR, y el análisis de residuos nos muestra la gráfica de aciertos vs errores.

Modelos finales NBA

Modelos Ensamblados: homogéneas y heterogéneas

Instrucciones: Se generan al menos cuatro modelos cubriendo ambas estrategias.

Decidimos utilizar los árboles que ya habían presentado un rendimiento aceptable y un stacking que nos ayuda a utilizar los mejores modelos individuales.

Para las 3 variables objetivo (PTS, REB y AST) se decidió usar los 4 modelos de ensamblaje.

```
param_grids = {
    # Hiperparámetros para RandomForest
    'RandomForest': {
        'regressor__n_estimators': [50, 100, 200],
        'regressor__max_depth': [None, 5, 10, 15],
        'regressor__min_samples_split': [2, 5, 10],
        'regressor__max_features': ['sqrt', 'log2', None]
    },

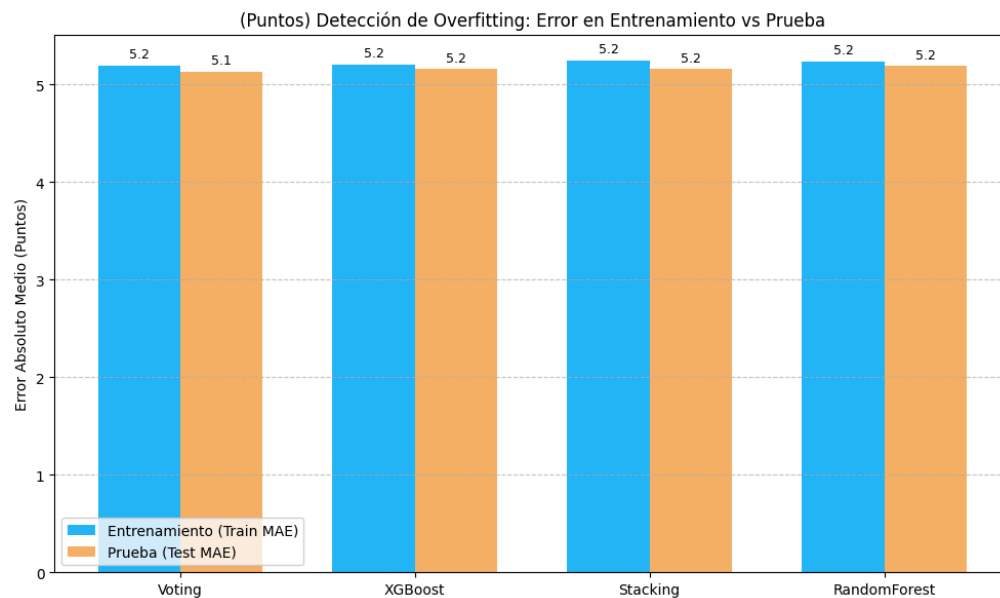
    # Hiperparámetros para XGBoost
    'XGBoost': {
        'regressor__n_estimators': [50, 100, 200],
        'regressor__learning_rate': [0.01, 0.05, 0.1],
        'regressor__max_depth': [3, 5, 7],
        'regressor__subsample': [0.8, 1.0]
    },

    # Hiperparámetros para VotingRegressor
    # Ajustamos los pesos que se le da a cada modelo base (Ridge, SVR, Tree)
    'Voting': {
        'regressor__weights': [
            None,          # Todos pesan igual
            [2, 1, 1],     # Se le da más peso a Ridge
            [1, 2, 1],     # Se le da más peso a SVR
            [1, 1, 2]      # Se le da más peso al Árbol
        ]
    },

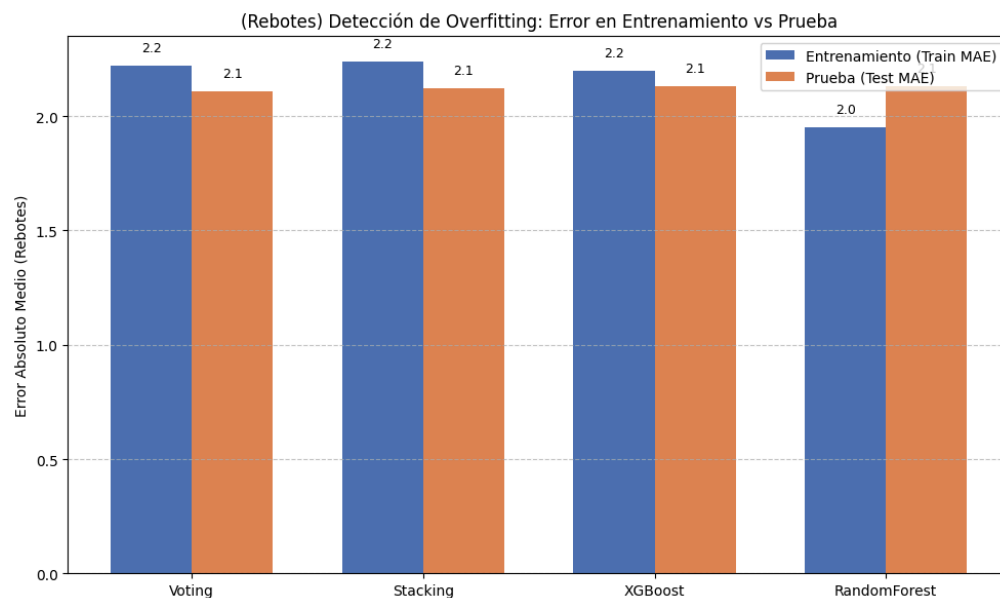
    # Hiperparámetros para StackingRegressor
    # Ajustamos la regularización del metamodelo (Ridge) y parámetros de un modelo base
    'Stacking': {
        'regressor__final_estimator__alpha': [0.1, 1.0, 10.0],
        'regressor__svr__C': [0.1, 1.0, 10.0], # Ajustando el SVR interno
        'regressor__tree__max_depth': [3, 5, 7] # Ajustando el árbol interno
    }
}
```

A continuación, se muestran las gráficas de resultados, en las mismas podemos ver un ligero sobre ajustes en los 4 modelos, más adelante descubriremos que esto es debido a que el modelo tiende a fallar más en la predicción de puntajes altos, sobre todo en la variable de puntos, debido a que hay menos partidos donde un jugador logre esa cantidad de puntos. Esto va a ser necesario considerarlo más adelante para ajustar las predicciones.

PTS



REB



AST

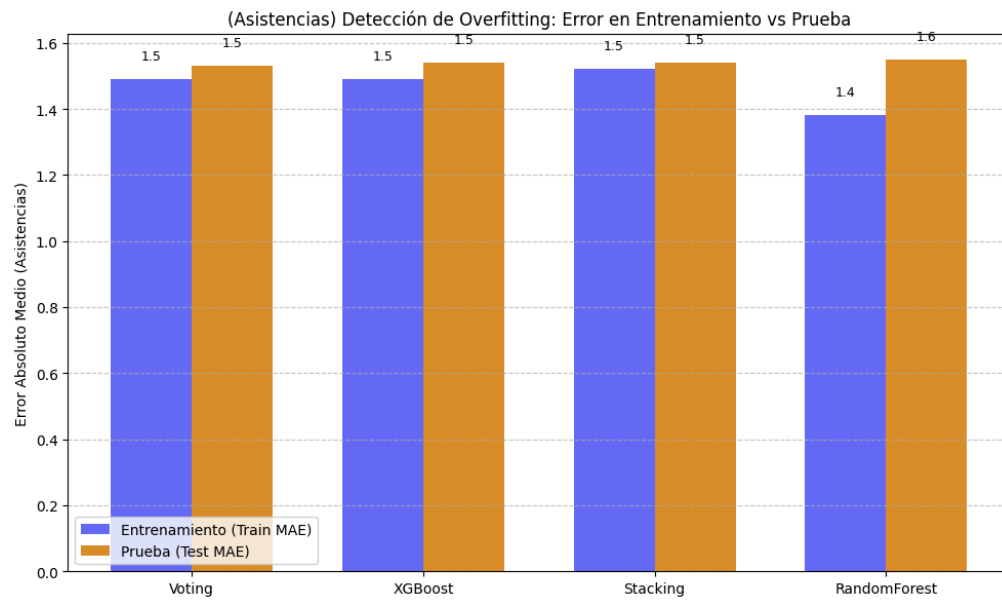


Tabla comparativa de modelos

A continuación, se muestran los resultados de los modelos ensambles y los modelos de la entrega anterior.

La tabla está organizada por el mejor MAE obtenido.

PUNTOS

Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo s
SVR	5.26	5.12	6.67	0.54	1457.84
Voting	5.20	5.13	6.63	0.54	95.53
Red neuronal	5.21	5.16	6.64	0.54	1680.20
XGBoost	5.21	5.16	6.64	0.54	25.49
Stacking	5.25	5.16	6.66	0.54	1534.29
Random Forest	5.24	5.19	6.70	0.54	544.20
Árbol	5.31	5.24	6.77	0.53	5.43

REBOTES

Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo s
SVR	2.29	2.11	2.75	0.38	397
Voting	2.22	2.11	2.73	0.39	91.19
Stacking	2.24	2.12	2.75	0.38	1611.22
XGBoost	2.20	2.13	2.75	0.38	23.15
Random Forest	1.95	2.13	2.75	0.38	563.65
Red neuronal	2.24	2.14	2.75	0.38	753.45
KNN	2.18	2.15	2.8	0.36	26.58

ASISTENCIAS

Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo s
Red neuronal	1.51	1.52	2.05	0.46	653.75
Voting	1.49	1.53	2.06	0.46	94.05
SVR	1.53	1.53	2.09	0.45	447.26
XGBoost	1.49	1.54	2.06	0.46	23.09
Stacking	1.52	1.54	2.06	0.46	1680.16
Random Forest	1.38	1.55	2.07	0.45	526.13
Lasso	1.55	1.55	2.07	0.46	0.36

Elección del modelo final

Tenemos un dilema complicado, ya que tenemos que irnos al segundo decimal de nuestro MAE (¿en promedio cuánto se equivocan los resultados?) para decidir nuestro mejor modelo, y poder ajustar aún más los hiperparámetros con ayuda de la librería de optuna para esto seguimos los siguientes pasos:

1) Agregar y eliminar columnas:

Nuestro MAE más alto es para PTS, es la variable a predecir que más nos preocupa poder bajar el error de las predicciones, por lo tanto, decidimos escarbar en la API de la NBA para ver si podíamos encontrar variables que pudieran ayudarnos a predecir el ritmo del partido y la eficiencia del jugador.

Se decidió agregar el promedio de los últimos 5 partidos de las siguientes medidas al conjunto de datos:

- **Porcentaje de tiros verdaderos (True shooting):** Es una fórmula que nos ayuda a medir la eficiencia de tiro combinando tiros de 2 pts, 3 pts y tiros libres. De esta forma ya no tenemos que pasar los por separado, si no que combinamos todo en una sola medida.
- **USG (Usage rate):** Nos indica el porcentaje de las jugadas del equipo que finaliza el jugador cuando está en cancha, es decir, de las jugadas de equipo cuántas las termina lanzando el jugador objetivo.
- **PACE del juego:** Es el ritmo del juego, indica las posesiones por partido que se permiten en el juego, un número alto indica que hay más posesiones, por lo tanto, más oportunidades de anotar.

Estas variables nos permiten eliminar ciertas columnas para no saturar el entrenamiento de los modelos con datos que no aportan mucho a la predicción, nuestras columnas objetivo quedaron de la siguiente manera:

```

features_input_5_matches_mean = [
    # Básicos
    'MIN_L5_AVG',          # Oportunidad: Minutos de juego

    # Tiros (volumen + eficiencia)
    'FGA_L5_AVG',          # Volumen: Tiros de campo intentados
    'TS_L5_PCT',           # Eficiencia: Porcentaje de tiros anotados

    'USG_PCT_AVG',         # Protagonismo: Porcentaje de las jugadas finaliza
    'OPP_PACE_AVG',        # Entorno: Ritmo de juego
    'PTS_SEASON_AVG',      # Ancla estadística: promedio de pts de la temporada

    # Contexto del partido
    'IS_HOME',             # ¿Juegan de local?
    'DAYS_REST',           # Días de descanso

    # Otras estadísticas
    'PF_L5_AVG',           # Faltas personales (Personal Fouls)
    'PLUS_MINUS_L5_AVG',   # Diferencia de puntos del equipo cuando el jugador está en cancha

    # Promedios históricos
    '#H2H_PTS_AVG',
    'H2H_REB_AVG',
    '#H2H_AST_AVG',

    # Datos de los rivales
    'OPP_FG_PCT_ALLOWED_AVG',
    '#OPP_PTS_ALLOWED_AVG',
    'OPP_REB_ALLOWED_AVG',
    '#OPP_AST_ALLOWED_AVG',
]

```

2) Juego con los hiperparámetros:

Nuestro segundo criterio fue escoger un modelo con el cual podamos tener más juego a la hora de mover hiperparámetros, para este escenario una red neuronal es la principal candidata, por su versatilidad podemos ajustar desde cuántas capas ocultas necesitamos hasta el tipo de pérdida para mejorar la retropropagación.

3) Comparación de métricas usando ajuste de hiperparámetros con Optuna

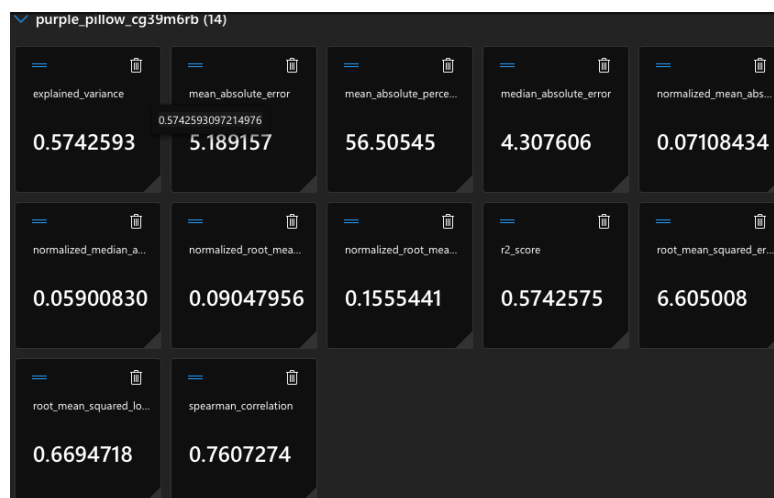
Decidimos agarrar nuestra primera variable objetivo (PTS) para ver si los dos primeros criterios podrían darnos mejores resultados en un red neuronal que con un SVR o un XGBoost, incluso decidimos usar los modelos ensambles para hacer esta batalla un poco más justa.

Modelo	Número de intentos	Test MAE
Red neuronal	20	4.79
SVR	20	4.82
XGBoost	20	4.84
Voting	Grid Search clásico	4.84
XGBoost	Grid Search clásico	4.85
Stacking	Grid Search clásico	4.87
Random forest	Grid Search clásico	4.89

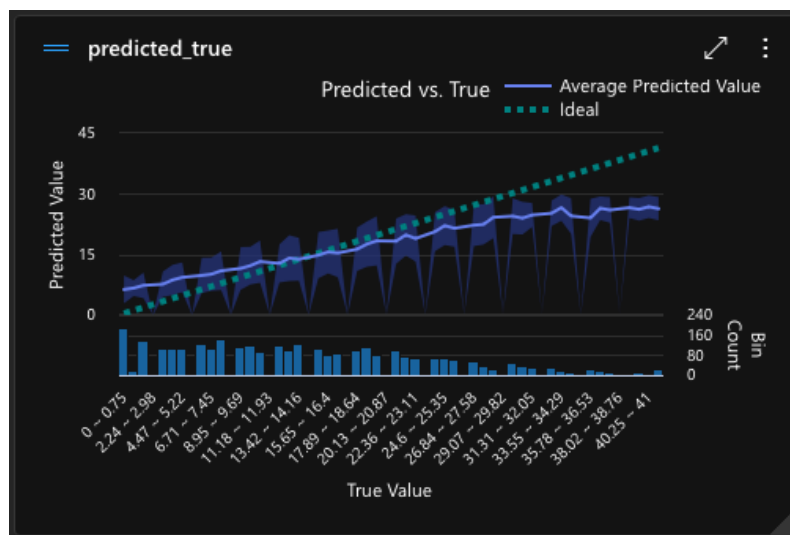
A pesar de ver unas mejoras relativamente buenas en todos los modelos (ya que se logró bajar del umbral de 5.1 MAE) vemos que el único modelo que logró bajar el umbral de 4.8 fue la red neuronal.

Incluso para estar seguros de nuestras combinaciones decidimos apoyarnos de la herramienta Azure Machine Learning, la misma nos ayuda a entrenar modelos con tan solo subir nuestro dataset.

Ninguno de los 30 modelos (limitados a 50 intentos) superó a nuestra red neuronal, la mayoría tuvo un MAE cercano a 5.2 o 5.1.



También rescatamos la gráfica generada por Azure ML de los valores predichos vs los valores reales, que vamos a reutilizar en la siguiente sección, la misma es muy interesante de analizar, ya que podemos observar que donde más falla nuestro modelo es a la hora de predecir valores altos en los puntos, concretamente podemos observar cómo a partir de los 25 pts comienza a separarse la línea morada de la línea punteada verde (la cual es la pendiente ideal que nuestras predicciones tendrían que seguir).



Decidimos probar con la red neuronal para la variable objetivo de puntos y además decidimos probar con XGBoost para las otras dos variables objetivo (rebotes y asistencias)

Red neuronal:

- Debido a la versatilidad que nos puede dar para ajustar toda la red podemos mejorar los valores de MAE, esto combinándolo con Optuna nos orilla a escoger el modelo para explorar muchas posibilidades que nos den un mejor resultado.
- Su tiempo de entrenamiento es intermedio y además podemos ver con cada retropropagación cual fue su su valor de pérdida, pudiendo identificar si hay un sobre/sub-entrenamiento.

- Los otros modelos estaban bastante cercanos a sus valores óptimos, la ventaja de una red neuronal, es que una capa oculta o una activación diferente puede darnos un mejor rendimiento.

XGBoost:

- Realmente en REB y AST se posicionó en el cuarto lugar con diferencias decimales respecto a los demás modelos, por lo tanto, tiene un buen desempeño y sobre todo su entrenamiento es muy rápido. Según la tabla de la sección anterior, este modelo fue el más rápido en entrenarse, por lo que vale la pena explorarlo y literalmente “no perdemos tiempo”.

Gráficos: Análisis final del rendimiento del modelo final

Los resultados de los entrenamientos fueron los siguientes:

Nota: Cabe aclarar que estos hiperpámetros encontrados con optuna y posteriormente entrenados fueron considerando las nuevas variables agregadas:

- Porcentaje de tiros acertados (True shooting).
- USG: Porcentaje de las jugadas finalizadas por el jugador.
- PACE: Ritmo del partido (posesiones por partido).
- PTS: Promedio de puntos en la temporada.
- H2H: Se dejaron nada más las variables correspondientes al objetivo, es decir, para pts se dejó H2H_pts para reb se dejó H2H_reb y para ast H2H_ast.
- OPP: Lo mismo que H2H.
- Para **pts y reb se aplicó log1p** a las variables objetivos, esto para estabilizar la varianza, corregir la asimetría de los datos y manejar relaciones no lineales
 - Ast mostró un desempeño peor al intentar aplicar una transformación logarítmica.

Variable objetivo	Modelo	Test MAE Optuna	Test MAE Final	R2	log1p aplicado	Tiempo s
Puntos	Red neuronal	5.1	4.63	0.38	Sí	179.92
Rebotes	Red neuronal	2.17	2.11	0.29	Sí	186.48
Asistencias	Red neuronal	1.68	1.74	0.48	No	95.20
Asistencias	XGBoost	1.63	1.74	0.48	No	9.09

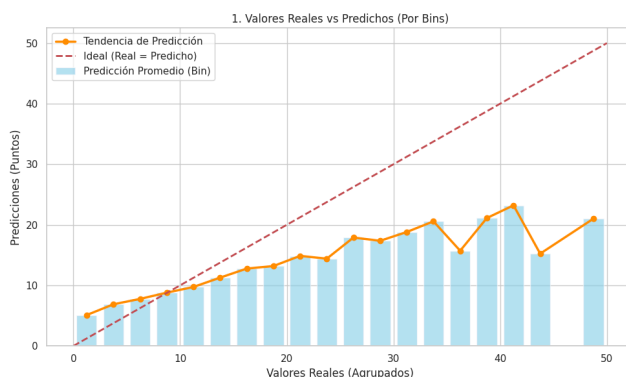
En asistencias vamos a tomar como modelo final la red neuronal, ya que nos dio exactamente el mismo resultado en Test MAE que XGBoost, por lo tanto, las gráficas se pueden explicar en conjunto si todas las variables objetivo usan una red neuronal.

Gráfico 1 Valores reales vs predichos (BINS):

Como se comentó en páginas anteriores, este gráfico intenta replicar el que muestra Azure Machine Learning, nos gustó por dos razones:

1. La línea roja muestra la pendiente “ideal” (casi fantástica) que deberían seguir nuestras predicciones.
2. Podemos comparar la segunda línea (en este caso la naranja) para poder observar qué tan acertadas son las predicciones y sobre todo cuál es el punto donde comienza a romperse. Particularmente podemos observar en las 3 variables como le cuesta predecir números grandes, esto va a terminar siendo una constante y un dolor de cabeza, dado que no hay muchos datos de jugadores que hagan más de 30 puntos, 15 rebotes y 10 asistencias en un partido de la NBA
3. Al agrupar por bins y no por nube de puntos es más fácil seguir la tendencia de la línea y ver como en los valores bajos tiende a estar cerca de la línea punteada roja.

PTS



REB (gráfico izquierdo) & AST (gráfico derecho)

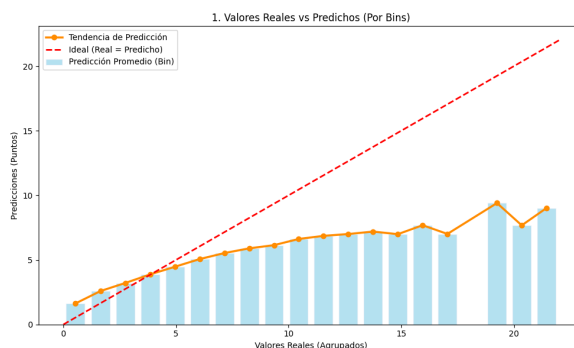
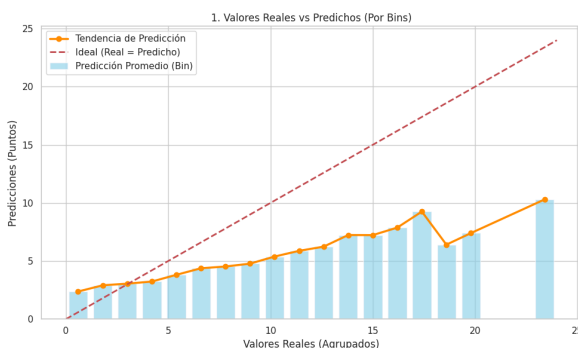
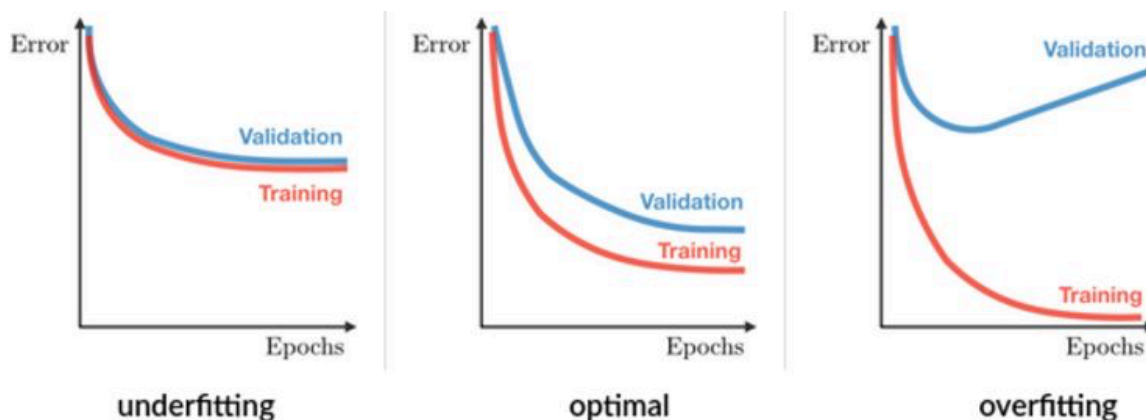


Gráfico 2 Curva de aprendizaje (diagnóstico de sobre/sub-ajuste):

La siguiente gráfica nos ayuda a verificar que nuestro modelo no esté sobre-entrenado o sub-entrenado, buscamos que ambas curvas (entrenamiento y validación) bajen al mismo tiempo y tiendan a converger en un mismo punto.

El siguiente ejemplo muestra mejor nuestro objetivo:

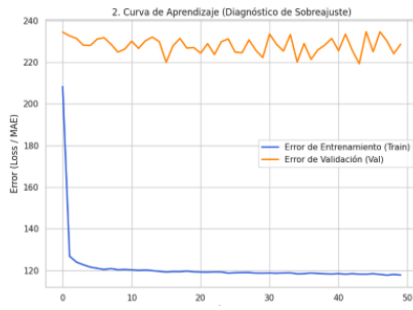


Algo que hay que destacar, es que las gráficas de puntos y rebotes pueden caer en el fenómeno conocido como “gráfico truncado”, ocurre cuando uno de los ejes de un gráfico (generalmente el eje vertical) no comienza en cero, sino en un valor superior, es una técnica útil que permite hacer zoom en variaciones, pero frecuentemente se utiliza de manera engañosa para exagerar diferencias o alteraciones de los datos.

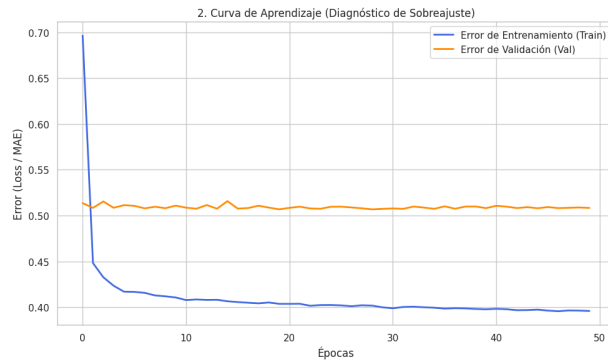
Tomando en cuenta lo anterior, podría parecer que nuestros modelos de puntos y rebotes caen en un overfitting, sin embargo es todo lo contrario, al querer hacer zoom en la gráfica caímos en el fenómeno mencionado anteriormente, para validar nuestro punto, presentamos una gráfica de un modelo de puntos donde sí se nota un claro sobre-ajuste del modelo.

El modelo de asistencias sí cabe recalcar que tuvo un mejor desempeño con los datos de validación, probablemente había asistencias con valores más bajos, es decir, datos más fáciles de predecir, al ser una separación pequeña podemos continuar con este modelo.

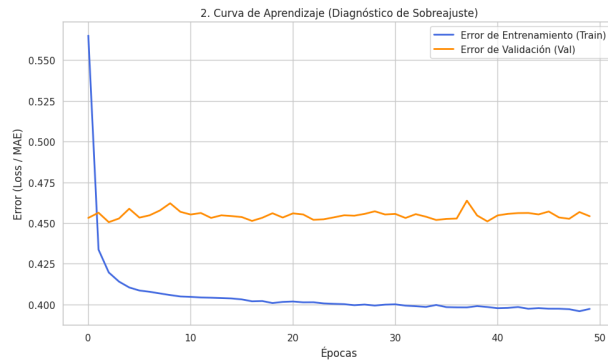
Modelo de PTS con un sobre-ajuste severo



PTS



REB



AST

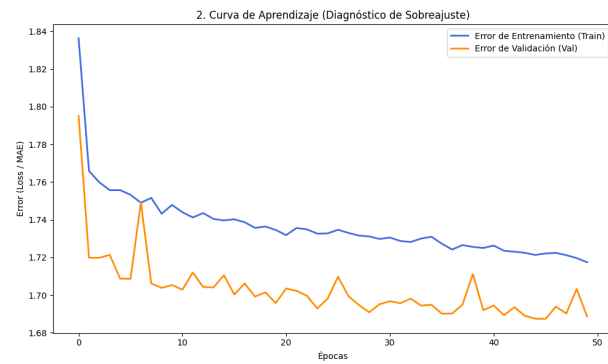


Gráfico 3 Análisis de residuos:

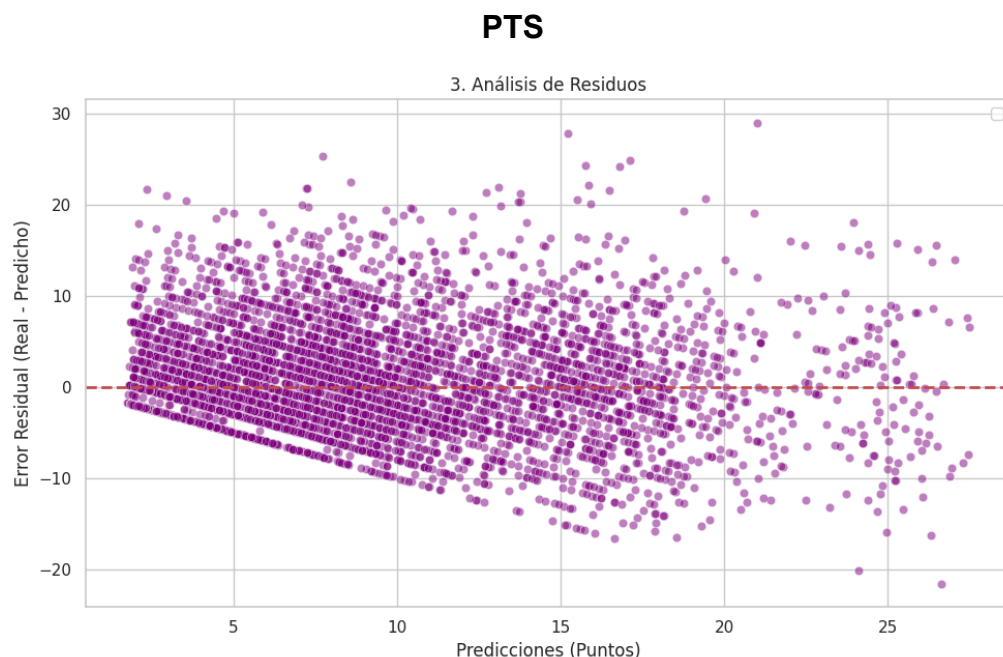
Estos son los gráficos que peor salieron, donde muestran (otra vez) una clara caída del desempeño del modelo al intentar predecir valores grandes en pts, reb y ast.

Este gráfico evalúa la diferencia entre los valores reales y los valores predichos por el modelo (residuo = valor real - valor predicho). El objetivo precisamente es observar si el modelo ha captado información o si persisten errores y sesgos.

Lo ideal es ver una “nube” aleatoria alrededor de la línea roja central (la que se encuentra en el número 0). Si se observa un patrón claro (ej. un embudo), el modelo tiene problemas con números altos.

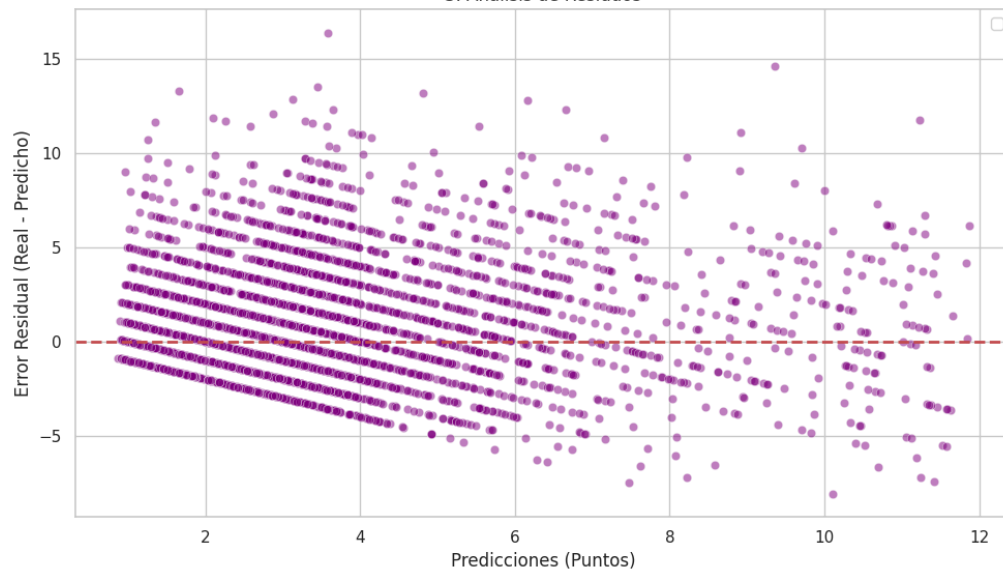
No está indicando que la varianza de los errores no es constante. Nuestros modelos hacen buenas predicciones en algunos rangos pero falla en otros.

Nota para el futuro: Dado que ya añadimos transformaciones logarítmicas a las variables objetivo, se podría intentar añadir polinomios, o intentar con modelos un poco más complejos (¿hay algo más complejo que una red neuronal?).



REB

3. Análisis de Residuos



AST

3. Análisis de Residuos

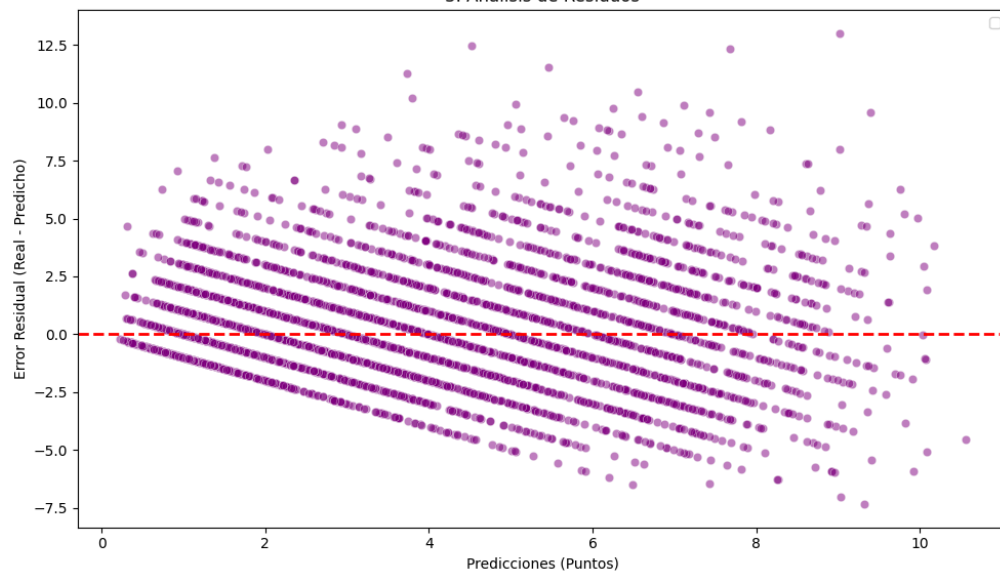


Gráfico 4 Histograma de residuos:

Este gráfico muestra la distribución de los errores de predicción de nuestros modelos. Lo que se busca comprobar, es que el modelo esté haciendo buenas predicciones de manera imparcial.

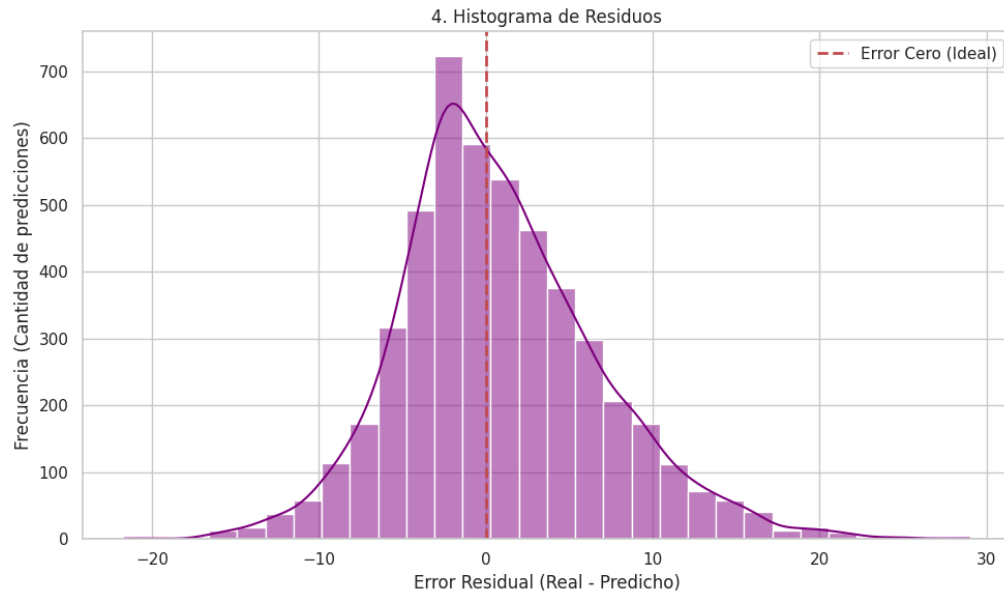
Lo ideal es ver una forma de “campana” (distribución normal) exactamente en la línea roja del 0.

- Si la campana tiene sesgo hacia la derecha: El modelo suele predecir menos (subestima).
- Si la campana tiene sesgo hacia la izquierda: El modelo suele predecir más (sobreestima).

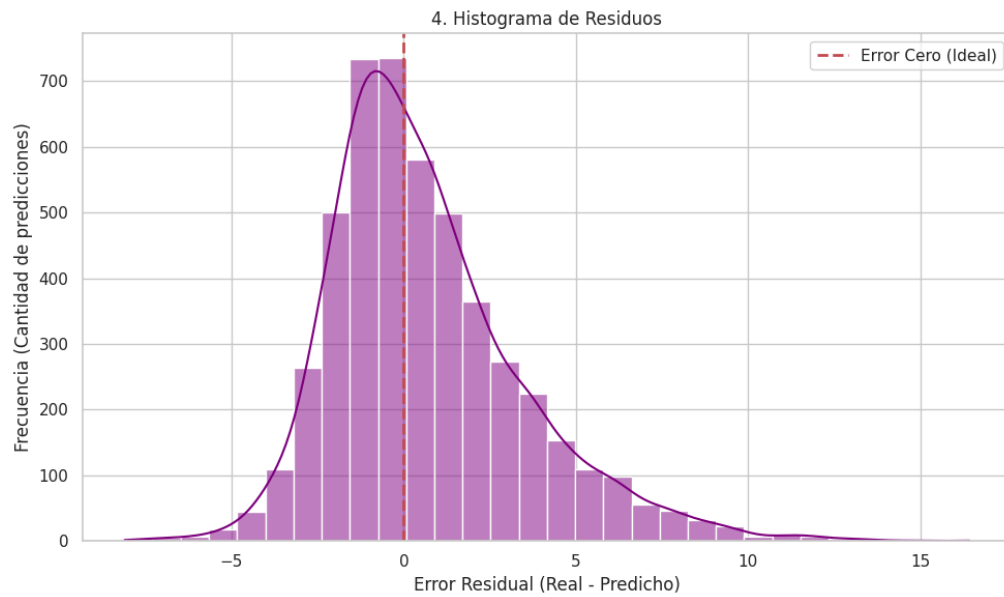
Podemos observar que en los tres gráficos, se acerca mucho a la forma acampanada y sobre todo a la zona central del gráfico, no obstante, en los tres observamos problemas de heterocedasticidad, es decir, la cola de las campanas tienen sesgo hacia la derecha, subestimando sobre todo los valores altos.

Los resultados de los gráficos precisamente los podemos relacionar con los gráficos 1 y 3, donde ya observamos que el modelo tiende a predecir valores más bajos que los reales, sobre todo cuando se trata de predecir valores altos en puntos, rebotes y asistencias (un buen partido de Lebron, Jokic o Wembanyama, por ejemplo).

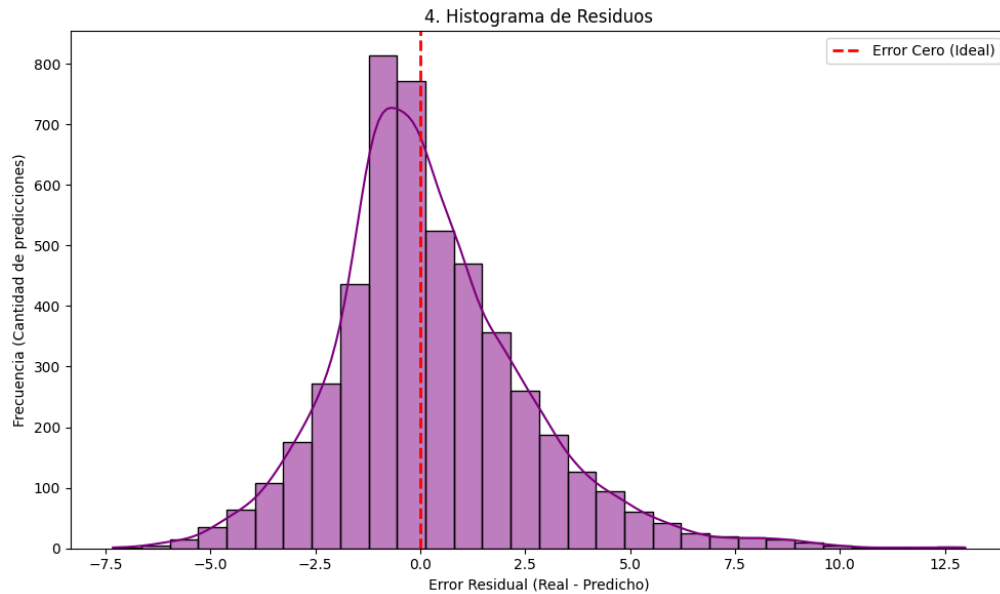
PTS



REB



AST



Conclusiones

Observando las mejoras realizadas por la búsqueda de hiperparámetros con optuna y analizando las gráficas, creemos que tenemos unos primeros modelos decentes, sobre todo considerando los tiempos apretados del proyecto.

Además, todavía podemos interpretar estos datos agregando una simulación Monte Carlo por ejemplo y claro, no hay que olvidar que el factor humano siempre tiene un toque impredecible.

Podemos buscar otras soluciones para mejorar los modelos, el primer paso que haría en el futuro, sería recolectar más datos “anómalos”, es decir, partidos donde haya muchos pts, reb y ast por parte de un jugador, esto con el objetivo de que el modelo tenga más ejemplos para aprender.

Pero tenemos modelos para poder seguir con la segunda fase del proyecto, interpretar los datos y hacerlo accesible al público.

Referencias

Leidwein, F. (2025). *How to Build an ELO Rating System for the Premier League in Python*. Recuperado de: <https://statsultra.com/how-to-build-an-elo-rating-system-for-the-premier-league-in-python/>

Amazon Web Services. (2024). Amazon Comprehend Developer Guide. <https://aws.amazon.com/comprehend/>

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

Galván Bellman, J. C., Méndez González, L. C., Pérez Olguín, I. J. C., & Quezada Carreón, A. E. (2024). Algoritmo de Machine Learning para la Predicción de un Partido de Fútbol. *Revista Academia Journals*, 16(4).

GDELT Project. (2024). Global Database of Events, Language, and Tone. <https://www.gdeltproject.org/>

Google Cloud. (2024). Natural Language API Documentation. <https://cloud.google.com/natural-language>

Maher, M. J. (1982). Modelling association football scores. *Statistica Neerlandica*, 36(3), 109-118.

Microsoft Azure. (2024). Text Analytics Documentation. <https://azure.microsoft.com/en-us/products/ai-services/text-analytics/>

NewsAPI. (2024). News API Documentation. <https://newsapi.org/>

Visengeriyeva, L., Kammer, A., Bär, I., Kniesz, A., y Plöd, M. (2023). *CRISP-ML(Q). The ML Lifecycle Process*. MLOps. INNOQ. <https://ml-ops.org/content/crisp-ml>